Veri Bilimci Müfredatı 2026 | Modül 46

PORTFOLIO & GITHUB

Ayrıntılı Konu Anlatımı

Modül 46 - Veri Bilimci Müfredatı 2026

Bu modülü bitirdiğinde

Portföyünü rastgele notebook koleksiyonundan çıkarıp; problemi, veriyi, kodu, doğrulamayı, sonucu ve tekrar üretilebilirliği açıkça gösteren profesyonel GitHub projelerine dönüştürebileceksin. Bir işe alım uzmanının veya teknik mülakatçının repoyu birkaç dakika içinde anlayabileceği yapı kurmayı öğreneceksin.

Kapsam: portföy stratejisi, proje seçimi, Git/GitHub temelleri, repo mimarisi, README, reproducibility, environment yönetimi, notebook düzeni, src yapısı, data/versioning, test ve kalite, commit/branch/PR, dokümantasyon, demo/deployment, profil optimizasyonu, örnek proje şablonları ve kontrol listeleri.

İçindekiler

- 1. Data Science Portföyünün Amacı
- 2. İşe Alımcı Bir Portföye Nasıl Bakar?
- 3. Portföy Stratejisi ve Proje Seçimi
- 4. Git ve GitHub Temelleri
- 5. Profesyonel Repository Mimarisi
- 6. README Tasarımı
- 7. Reproducibility: Proje Başkasında Çalışmalı
- 8. Environment ve Dependency Yönetimi
- 9. Notebook ve src/ Kod Organizasyonu
- 10. Veri Yönetimi, Gizlilik ve .gitignore
- 11. Test, Kod Kalitesi ve Otomasyon
- 12. Git Workflow: Commit, Branch ve Pull Request
- 13. Issues, Project Board ve Release Mantığı
- 14. Dokümantasyon ve Görsel Sunum
- 15. Demo, API ve Deployment
- 16. GitHub Profilini Portföye Dönüştürmek
- 17. Proje Case Study Yazımı
- 18. E-Commerce Projesi için Örnek Repo
- 19. Örnek README Şablonu
- 20. Portföy Anti-Patternleri
- 21. Teknik Değerlendirme Rubriği
- 22. 30 Günlük Portföy İnşa Planı
- 23. Mülakatta GitHub Projesini Anlatmak
- 24. Final Kontrol Listesi

Temel prensip

Portföyün amacı "çok şey yaptım" demek değildir. Amaç: gerçek bir problemi nasıl tanımladığını, veriyi nasıl işlediğini, teknik kararları neden verdiğini ve çalışmanın yeniden üretilebilir olduğunu kanıtlamaktır.

1. Data Science Portföyünün Amacı

Bir veri bilimci portföyü, CV'deki iddiaların teknik kanıtıdır. İyi bir repo yalnızca final modeli göstermez; problem tanımını, veri kaynaklarını, veri kalitesi sorunlarını, baseline yaklaşımını, deneyleri, değerlendirme metriklerini, hata analizini ve sonucun nasıl kullanılacağını da gösterir.

1.1 Portföy hangi sorulara cevap vermeli?

- Bu kişi gerçek bir problemi analitik probleme çevirebiliyor mu?
- SQL/Python ile temiz ve anlaşılır veri işleme yapabiliyor mu?
- EDA sonucunda anlamlı hipotezler kuruyor mu?
- Modeli neden seçtiğini ve alternatifleri açıklayabiliyor mu?
- Validation, leakage ve metric seçimini biliyor mu?
- Kod başka bir bilgisayarda tekrar çalıştırılabilir mi?
- Sonuçları teknik olmayan kişilere anlatabiliyor mu?
- Proje üretim ortamına taşınsa ne değişeceğini düşünüyor mu?

1.2 Portföy ≠ proje mezarlığı

Kalite > sayı

20 yarım notebook yerine 4-6 güçlü, açıklanabilir ve yeniden üretilebilir proje daha iyi sinyal verir. Her proje farklı bir yeteneğini göstermeli.

2. İşe Alımcı Bir Portföye Nasıl Bakar?

Bir repo teknik olarak çok iyi olsa bile ilk bakışta anlaşılmıyorsa değerinin büyük kısmı görünmez kalır. Portföy katmanlı okunabilir olmalıdır: ilk 30 saniyede amaç, birkaç dakikada yöntem ve sonuç, daha derin incelemede kod kalitesi görülebilmelidir.

İnceleme seviyesi	Bakan kişi	Aradığı sinyal
30 saniye	Recruiter / hiring manager	Proje adı, problem, stack, sonuç, görsel
2-5 dakika	Data Scientist	EDA, metric, model seçimi, sonuçların mantığı
10+ dakika	Technical interviewer	Kod yapısı, validation, test, reproducibility, trade-off
Deep dive	Takım lideri / ML Engineer	Architecture, deployment, monitoring, ölçekleme

2.1 İlk ekran testi

- Repo adı açıklayıcı mı?
- README ilk paragrafı problemi söylüyor mu?
- Bir sonuç özeti var mı?
- Kurulum ve çalıştırma komutu görünüyor mu?
- Klasör yapısı anlaşılır mı?
- Notebook dışında düzenli kod var mı?

3. Portföy Stratejisi ve Proje Seçimi

Portföy projeleri birbirinin kopyası olmamalıdır. İdeal olarak farklı problem tiplerini ve çalışma biçimlerini kapsar.

Proje tipi	Gösterdiği yetenek	Örnek
Analytics	SQL, KPI, dashboard, storytelling	E-commerce funnel analizi
Supervised ML	Feature engineering, validation, model evaluation	Churn prediction
Unsupervised	Segmentasyon ve yorumlama	Customer segmentation
Time Series	Temporal validation, forecasting	Demand forecasting
NLP / LLM	Text pipeline, embeddings, retrieval	Review analysis / RAG
Production ML	API, Docker, monitoring mantığı	Fraud scoring API

3.1 Proje seçme puanlama sistemi

Kriter	0 puan	1 puan	2 puan
Business problem	Belirsiz	Kısmen açık	Net karar problemi
Data	Oyuncak veri	Gerçekçi veri	Gerçek dünya karmaşıklığı
Teknik derinlik	Tek notebook	Birden fazla analiz	Pipeline + test + serving
Anlatılabilirlik	Sonuç belirsiz	Temel sonuç	Net insight + trade-off
Farklılaşma	Tutorial kopyası	Uyarlanmış	Kendi hipotez/kararların

Seçim kuralı

Bir proje yeni bir yetenek göstermiyorsa portföye eklenmesi şart değildir. Aynı şeyi yapan beş classification projesi yerine farklı problem tipleri daha değerlidir.

4. Git ve GitHub Temelleri

4.1 Git nedir?

Git, dosyaların geçmişini takip eden dağıtık bir sürüm kontrol sistemidir. GitHub ise Git repository'lerini barındıran, işbirliği ve proje yönetimi özellikleri sunan platformdur. Git değişiklik geçmişini; GitHub paylaşım ve işbirliği katmanını sağlar.

4.2 Temel komut akışı

```
git clone <repo-url>
git status
git add .
git commit -m "feat: add churn feature pipeline"
git pull --rebase
git push origin main
```

4.3 En sık kullanılan komutlar

Komut	Amaç
git init	Yeni repository başlatır
git clone	Remote repository kopyalar
git status	Değişiklikleri gösterir
git add	Commit için staging yapar
git commit	Değişiklik snapshotı oluşturur
git log	Geçmişi görüntüler
git diff	Değişiklik farkını gösterir
git branch	Branch yönetir
git switch	Branch değiştirir
git pull	Remote değişiklikleri getirir
git push	Local commitleri remote'a gönderir

Commit mantığı

Commit bir "kaydet" düğmesi değildir. Anlamlı ve tek bir işi temsil eden küçük değişiklik paketidir. "final", "fix", "son2" gibi mesajlar geçmişi değersizleştirir.

5. Profesyonel Repository Mimarisi

Data science projelerinde klasör yapısı projenin zihinsel modelidir. Veri, notebook, üretim kodu, model artefactları, test ve dokümantasyon birbirinden ayrılmalıdır.

```
ecommerce-analytics/
├── README.md
├── LICENSE
├── .gitignore
├── pyproject.toml          # veya requirements.txt
├── Makefile
├── configs/
│   └── config.yaml
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_engineering.ipynb
│   └── 03_model_evaluation.ipynb
├── src/
│   ├── data/
│   │   └── make_dataset.py
│   ├── features/
```

```
│   │   └── build_features.py
│   ├── models/
│   │   ├── train.py
│   │   └── predict.py
│   └── visualization/
│       └── plots.py
├── tests/
│   ├── test_features.py
│   └── test_metrics.py
├── reports/
│   └── figures/
└── models/
    └── .gitkeep
```

5.1 Klasörlerin sorumluluğu

Klasör	İçerik	Kural
data/raw	Kaynak veri	Mümkünse değiştirilmez
data/processed	Model/analiz girdisi	Pipeline tarafından üretilir
notebooks	Keşif ve anlatımlı analiz	Numaralı ve temiz tutulur
src	Tekrar kullanılabilir kod	Fonksiyon/modül halinde
tests	Otomatik testler	Kritik dönüşümleri korur
configs	Parametreler	Kod içine gömülmez
reports	Grafik ve raporlar	README tarafından kullanılabilir
models	Model artefactları	Büyük dosyalar Git'e konmaz

6. README Tasarımı

README, projenin ana sayfasıdır. Bir ziyaretçi repository'yi klonlamadan önce projenin ne yaptığını ve neden önemli olduğunu buradan anlamalıdır.

6.1 İdeal README sırası

1. Proje adı ve tek cümlelik değer önerisi.
2. Problem / business context.
3. Temel sonuçlar ve önemli metrikler.
4. Görsel veya dashboard ekran görüntüsü.
5. Dataset ve veri sözlüğü özeti.
6. Yaklaşım / methodology.
7. Repository yapısı.
8. Kurulum.
9. Çalıştırma komutları.
10. Sonuçlar, hata analizi ve limitation.
11. Sonraki adımlar.

6.2 Güçlü README açılışı

```
# Customer Churn Prediction

Bir abonelik işletmesinde churn riski yüksek müşterileri erken tespit etmek için
uçtan uca makine öğrenmesi pipeline'ı. Model yalnızca ROC-AUC optimize etmek yerine
retention ekibinin müdahale kapasitesi ve false-positive maliyeti dikkate alınarak
threshold bazında değerlendirilmiştir.

**Sonuç:** Recall@Top10% = 0.61, baseline = 0.34.
```

README kuralı

README'yi yalnızca "kodu nasıl çalıştırırım?" dokümanı gibi düşünme. Aynı zamanda projenin kısa teknik case study'sidir.

7. Reproducibility: Proje Başkasında Çalışmalı

Reproducibility, aynı kod ve aynı girdiyle benzer sonuçların yeniden üretilebilmesidir. Portföyde bu, profesyonellik sinyalidir çünkü işe alınan kişi yalnızca kendinde çalışan notebook değil, başkasının da çalıştırabileceği sistem üretmelidir.

7.1 Minimum reproducibility şartları

- Python sürümü belirtilmeli.
- Dependencies sabitlenmeli veya tanımlanmalı.
- Random seed kullanılmalı.
- Verinin nasıl elde edileceği açıklanmalı.
- Dosya yolları kişisel bilgisayara bağımlı olmamalı.
- Train/test split mantığı deterministik olmalı.
- Model/rapor üretme komutu belgelenmeli.
- Gerekli environment variable'lar .env.example içinde listelenmeli.

7.2 Tek komut hedefi

```
git clone <repo-url>
cd ecommerce-analytics
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m src.data.make_dataset
python -m src.models.train
```

Kendini test et

Projeyi yeni ve boş bir klasöre tekrar klonla. README dışında hiçbir bilgi kullanmadan çalıştırabiliyor musun? Hayırsa proje henüz reproducible değildir.

8. Environment ve Dependency Yönetimi

8.1 requirements.txt

```
pandas==2.3.2
numpy==2.2.6
scikit-learn==1.7.1
matplotlib==3.10.5
pytest==8.4.1
```

Tam sürüm pinlemek tekrar üretilebilirliği güçlendirir; ancak uzun ömürlü paketlerde kontrollü sürüm aralıkları veya lock dosyaları tercih edilebilir.

8.2 pyproject.toml yaklaşımı

```
[project]
name = "ecommerce-analytics"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
  "pandas>=2.2,<3",
  "scikit-learn>=1.6,<2",
  "pyyaml>=6,<7"
]

[project.optional-dependencies]
```

```
dev = ["pytest", "ruff"]
```

8.3 Environment variable

```
# .env.example
DATABASE_URL=postgresql://user:password@localhost:5432/db
MODEL_PATH=models/model.joblib
API_KEY=replace_me
```

Güvenlik

Gerçek API key, parola, token veya private connection string asla GitHub'a commit edilmemelidir. Secret sızdıysa yalnızca dosyayı silmek yetmez; secret rotate edilmelidir.

9. Notebook ve src/ Kod Organizasyonu

Notebook keşif, görselleştirme ve anlatım için çok uygundur; fakat tekrar kullanılan veri temizleme, feature engineering ve modelleme kodunun tamamının notebook hücrelerinde kalması bakım maliyetini artırır.

9.1 Notebook ne için?

- EDA ve hipotez geliştirme
- Grafik ve sonuç anlatımı
- Küçük deneyler
- Model karşılaştırmasının özetlenmesi

9.2 src/ ne için?

- Veri okuma/temizleme fonksiyonları
- Feature engineering
- Training ve prediction pipeline
- Metric hesapları
- Tekrar kullanılan plotting fonksiyonları
- Configuration ve logging

9.3 Kötü ve iyi örnek

```
# Kötü: notebook içinde 50 satırlık tekrar eden preprocessing

# İyi:
from src.features.build_features import build_customer_features
X = build_customer_features(df)
```

Notebook hijyeni

Notebook commit edilmeden önce hücreler baştan sona çalıştırılmalı, gereksiz debug çıktıları temizlenmeli ve execution order tutarlı olmalıdır.

10. Veri Yönetimi, Gizlilik ve .gitignore

10.1 Git'e ne konmamalı?

- Kişisel veya müşteri verisi
- Çok büyük raw datasetler
- Parola/token/.env
- Model binary'leri çok büyükse

- Cache ve virtual environment
- Notebook checkpointleri
- Yerel IDE ayarları

10.2 Örnek .gitignore

```
# Python
__pycache__/
*.py[cod]
.venv/

# Jupyter
.ipynb_checkpoints/

# Secrets
.env
*.pem

# Data / models
data/raw/*
data/processed/*
!data/**/.gitkeep
models/*.joblib
models/*.pkl

# OS / IDE
.DS_Store
.vscode/
```

10.3 Dataset paylaşım seçenekleri

Durum	Yaklaşım
Küçük açık veri	Repo içinde veya indirme scripti
Kaggle / açık platform	Kaynak linki + indirme talimatı
Büyük veri	Cloud/object storage veya DVC benzeri yaklaşım
Özel veri	Sentetik örnek + schema + üretme scripti
Lisans kısıtlı	Kaynağı belirt, dosyayı yeniden dağıtma

11. Test, Kod Kalitesi ve Otomasyon

Portfolio projesinde test yazmak, veri bilimi kodunu üretim kalitesine yaklaştıran güçlü bir sinyaldir. Her şeyi test etmek gerekmez; kritik dönüşümler ve hataya açık varsayımlar test edilmelidir.

11.1 Neleri test edebilirsin?

- Feature fonksiyonu beklenen kolonları üretiyor mu?
- Null oranı beklenen sınırda mı?
- Target leakage oluşturan kolonlar hariç tutuluyor mu?
- Metric fonksiyonu doğru hesaplıyor mu?
- Prediction çıktısı 0-1 aralığında mı?
- Schema değiştiğinde pipeline hata veriyor mu?

11.2 Pytest örneği

```
def test_discount_rate_is_bounded():
    df = sample_orders()
    result = build_features(df)
    assert result["discount_rate"].between(0, 1).all()

def test_target_not_in_features():
    X, y = make_training_data(sample_customers())
    assert "churned" not in X.columns
```

11.3 CI mantığı

```
# .github/workflows/tests.yml
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

Portföy sinyali
README üzerindeki yeşil test badge'i tek başına önemli değildir. Asıl sinyal, anlamlı testlerin gerçekten kritik veri ve model davranışlarını korumasıdır.

12. Git Workflow: Commit, Branch ve Pull Request

12.1 İyi commit mesajı

Zayıf	Daha iyi
update	docs: clarify model evaluation section
final	feat: add customer lifetime features
fix2	fix: prevent division by zero in margin feature
new notebook	analysis: add cohort retention exploration

12.2 Branch örneği

```
main
├── feature/customer-features
├── experiment/lightgbm-baseline
├── fix/date-parsing
└── docs/readme-results
```

12.3 Pull Request neden portföyde faydalı?

- Değişikliğin amacını açıklamayı öğretir.
- Code review düşünce yapısını gösterir.
- Değişiklikleri küçük parçalara ayırmayı sağlar.
- Takım çalışma pratiğini kanıtlar.

Tek kişilik projede bile
Kendi feature branch'inden main'e PR açmak yapay bir gösteri olmak zorunda değildir. Büyük değişiklikleri inceleme alışkanlığı kazandırır ve proje geçmişini okunabilir kılar.

13. Issues, Project Board ve Release Mantığı

13.1 Issue örnekleri

- BUG: datetime parsing fails for missing timezone

- FEATURE: add RFM segmentation
- EXPERIMENT: compare LightGBM and CatBoost
- DOCS: document raw data schema
- TECH DEBT: move preprocessing out of notebook

13.2 Issue şablonu

```
## Problem
Mevcut churn pipeline yeni customer_type değerinde hata veriyor.

## Expected behavior
Unknown kategori "other" olarak işlenmeli.

## Acceptance criteria
- Pipeline hata vermiyor
- Test ekleniyor
- README data assumptions güncelleniyor
```

13.3 Release/tag

Bir proje belirgin bir aşamaya geldiğinde v0.1.0, v1.0.0 gibi tag/release kullanmak; model, rapor veya demo sürümünü belirli bir commit ile ilişkilendirir.

14. Dokümantasyon ve Görsel Sunum

Data science portföyü görsel iletişim avantajına sahiptir. Ancak grafiklerin amacı repoyu süslemek değil, teknik hikâyeyi hızlandırmaktır.

14.1 README'de kullanılabilecek görseller

- Pipeline mimari diyagramı
- Dashboard ekran görüntüsü
- Confusion matrix / ROC / PR curve
- Feature importance / SHAP özeti
- Forecast actual vs predicted grafiği
- Segment profili
- API örnek request/response görüntüsü

14.2 Grafik kalite kontrolü

- Başlık bir mesaj taşıyor mu?
- Eksen adları ve birimler var mı?
- Renk sayısı gereksiz yüksek mi?
- Grafik iş sorusuna cevap veriyor mu?
- Görsel README'de okunabilecek çözünürlükte mi?

Storytelling
"ROC curve" başlığı yerine "Model, yüksek riskli müşterileri baseline'dan daha iyi sıralıyor" gibi sonuç odaklı bir anlatım daha güçlüdür; ancak iddianın ölçümünü hemen yanında göster.

15. Demo, API ve Deployment

Her projeyi deploy etmek gerekmez. Fakat en az bir portföy projesinde kullanıcıya açık demo veya API sunmak, modelin notebook dışına taşınabildiğini gösterir.

15.1 Seçenekler

Demo tipi	Ne gösterir?	Örnek

Dashboard	Analitik ve iletişim	Power BI / Tableau / Streamlit
Web app	Model etkileşimi	Streamlit / Gradio
REST API	Serving bilgisi	FastAPI
Docker image	Paketleme	API + model container
Batch CLI	Pipeline otomasyonu	python -m src.models.predict

15.2 FastAPI mini örnek

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Customer(BaseModel):
    tenure: int
    monthly_charges: float

@app.post("/predict")
def predict(customer: Customer):
    score = model.predict_proba([[customer.tenure, customer.monthly_charges]])[0, 1]
    return {"churn_probability": float(score)}
```

Demo güvenilirliği
Canlı demo çökerse README'de ekran görüntüsü, kısa GIF veya örnek çıktı bulunması projenin yine anlaşılmasını sağlar.

16. GitHub Profilini Portföye Dönüştürmek

16.1 Profil README

Kullanıcı adınla aynı isimde public repository oluşturarak profil README kullanabilirsin. Amaç teknoloji logosu duvarı kurmak değil; uzmanlık alanını, seçilmiş projeleri ve iletişim kanallarını hızlıca göstermektir.

16.2 Profilde olması gerekenler

- Kısa profesyonel tanım
- Ana yetkinlik alanları
- 3-5 pinned repository
- LinkedIn / portfolio / YouTube gibi ilgili bağlantılar
- Aktif olarak geliştirdiğin konu
- Tutarlı repo isimlendirmesi

16.3 Pinned repository stratejisi

Pin	Amaç
1	En güçlü end-to-end proje
2	SQL / analytics projesi
3	ML modeling projesi
4	Production/deployment projesi
5	Uzmanlaşmak istediğin alan: NLP/LLM/time series
6	Açık kaynak katkı veya araç projesi

17. Proje Case Study Yazımı

README ve mülakat anlatımı için aynı case study omurgasını kullanabilirsin.

17.1 Case study şablonu

12. Context: Şirket/kullanıcı problemi nedir?
13. Decision: Analiz veya model hangi kararı destekleyecek?
14. Data: Hangi veri var, kalite problemleri neler?
15. Baseline: Basit referans nedir?
16. Approach: SQL/EDA/model pipeline nasıl kuruldu?
17. Validation: Sonuç nasıl doğrulandı?
18. Result: Teknik sonuç ve business etkisi nedir?
19. Error analysis: Nerede başarısız oluyor?
20. Limitations: Hangi varsayımlar sonucu sınırlar?
21. Next steps: Production'da ne eklenir?

17.2 Sonuç cümlesi örneği

```
Model ROC-AUC=0.84 elde etti. Ancak operasyon ekibi müşterilerin yalnızca %10'una
ulaşabildiği için ana karar metriği olarak Recall@Top10% kullandım. Bu metrikte model
churn eden müşterilerin %61'ini yakalarken logistic regression baseline %34'te kaldı.
```

18. E-Commerce Projesi için Örnek Repo

E-Commerce Analytics projesini analist + data scientist yeteneklerini birlikte gösterecek şekilde aşağıdaki katmanlara ayırabilirsin.

Katman	Çıktı
SQL	KPI sorguları, cohort, funnel, top customers
EDA	Kategori, zaman, müşteri ve sipariş davranışı
Dashboard	Revenue, AOV, retention, conversion proxy
ML	Repeat purchase / churn / CLV tahmini
Engineering	src pipeline + tests + configs
Serving	İsteğe bağlı prediction API
README	Business story + sonuç + reproducibility

18.1 Önerilen repo

```
ecommerce-data-science/
├── README.md
├── sql/
│   ├── 01_kpis.sql
│   ├── 02_funnel.sql
│   └── 03_cohort.sql
├── notebooks/
│   ├── 01_business_eda.ipynb
│   ├── 02_customer_features.ipynb
│   └── 03_repeat_purchase_model.ipynb
├── src/
│   ├── data.py
│   ├── features.py
│   └── train.py
├── dashboard/
│   └── screenshots/
├── tests/
├── reports/figures/
└── requirements.txt
```

18.2 Ana README hikâyesi

- Şirket hangi sorunu çözmeye çalışıyor?
- Hangi KPI'lar hesaplandı?
- EDA'da hangi davranış farkları bulundu?

- Hangi segmentler değerli/riskli?
- ML modeli hangi aksiyonu destekliyor?
- Dashboard yöneticiye ne gösteriyor?
- Teknik limitation ve sonraki adım ne?

19. Örnek README Şablonu

````
# Project Name

One-line problem + value statement.

## Business Problem
What decision are we trying to improve?

## Key Results
- Metric 1: ...
- Metric 2: ...
- Business insight: ...

## Data
Source, size, target, important limitations.

## Methodology
1. Data validation
2. EDA
3. Feature engineering
4. Baseline
5. Model comparison
6. Final evaluation

## Repository Structure
```text
src/
notebooks/
tests/
reports/
```

## Reproduce
```bash
pip install -r requirements.txt
python -m src.models.train
```

## Results & Error Analysis
What worked? Where does the model fail?

## Limitations
Data, causality, drift, sample bias, etc.

## Next Steps
Production / monitoring / experiments.
````

Kopyala ama doldur

Bu şablon iskelet sağlar; tüm projelerde aynı genel yapı olabilir. Ancak "Results", "Limitations" ve "Decision" bölümleri projeye özgü olmalıdır.

20. Portföy Anti-Patternleri

Anti-pattern	Neden zayıf?	Düzeltme
Sadece notebook	Kod tekrar kullanılamaz	Tekrarlanan kodu src/ içine taşı
README yok	Proje anlaşılmıyor	Problem + sonuç + reproduce ekle
Tutorial kopyası	Karar yeteneği görünmez	Kendi hipotez ve deneyini ekle
Sadece accuracy	Business trade-off yok	Uygun metric + threshold açıkla
Test setine sürekli bakmak	Overfitting riski	CV/validation ile model seç
Dataset repo içinde 500 MB	Repo kullanışsız	Download script / external storage

Secret commit etmek	Güvenlik riski	.gitignore + secret rotation
20 teknoloji badge'i	Kanıt yerine dekorasyon	Projelerde gerçekten kullandığını göster
"Final.ipynb" / "final2.ipynb"	Süreç kaotik	Numaralı notebook + Git history

21. Teknik Değerlendirme Rubriği

Kendi projeni yayınlamadan önce aşağıdaki rubrikle değerlendir. Amaç puan peşinde koşmak değil, eksik sinyalleri görünür kılmaktır.

Alan	0	1	2
Problem framing	Amaç yok	Genel amaç	Karar ve kullanıcı net
Data quality	Yok	Temel temizlik	Schema, leakage, validation
EDA	Grafik yığını	Bulgular var	Hipotez ve aksiyon bağlantılı
Modeling	Tek model	Baseline + model	Doğru validation + trade-off
Code quality	Notebook karmaşası	Kısmi modülerlik	src + config + test
Reproducibility	Çalışmıyor	Kısmi talimat	Tekrarlanabilir pipeline
Documentation	Yetersiz	Temel README	Case study kalitesinde
Business impact	Yok	Metrik var	Karar/aksiyonla bağlı
Production thinking	Yok	Kısa not	Serving/monitoring/limitations

Hedef

Portföydeki her projenin her alanda maksimum olması gerekmez. Analytics projesi modeling'de daha hafif olabilir; production projesi ise engineering sinyalini güçlendirmelidir.

22. 30 Günlük Portföy İnşa Planı

Gün	Odak	Çıktı
1-3	Git/GitHub temeli	Repo, commit, branch pratiği
4-7	Proje 1 düzenleme	README + klasör yapısı + temiz notebook
8-11	Reproducibility	requirements/pyproject + scripts
12-14	Test ve kalite	pytest + lint + CI
15-18	Proje 2	ML case study repo
19-21	Model evaluation	Error analysis + limitation bölümü
22-24	Demo	Dashboard / Streamlit / API
25-26	GitHub profil	Profile README + pinned repos
27-28	Mülakat anlatımı	Her proje için 2-5 dk hikâye
29	Boş makinede test	Fresh clone + reproduce
30	Final audit	Link, görsel, typo, secret, lisans kontrolü

22.1 Her gün yapılabilecek mini Git pratiği

- Tek bir anlamlı değişiklik yap.
- git diff ile kontrol et.
- Açıklayıcı commit mesajı yaz.
- Feature branch kullan.
- PR açıklamasına "neden" kısmını ekle.

23. Mülakatta GitHub Projesini Anlatmak

23.1 2 dakikalık anlatım

```
1. Problem: "E-commerce şirketinde tekrar satın alma oranını artırmak istedim."
```

```
2. Data: "12 aylık sipariş ve müşteri davranış verisi kullandım."
3. Approach: "SQL ile KPI/cohort analizi, Python ile feature pipeline kurdum."
4. Model: "Logistic baseline sonrası gradient boosting karşılaştırdım."
5. Evaluation: "Temporal split kullandım; ana metric Recall@Top10%."
6. Result: "Baseline %31 iken final model %57 recall elde etti."
7. Limitation: "Kampanya maruziyeti verisi olmadığı için nedensellik iddia etmiyorum."
8. Production: "Batch skor üretimi + drift takibi eklerdim."
```

23.2 Mülakatçının devam soruları

- Neden bu dataset?
- En zor data quality sorunu neydi?
- Neden bu metric?
- Baseline neydi?
- Test setini nasıl korudun?
- En büyük hata segmentin hangisi?
- Bir ay daha süren olsa ne yapardın?
- Production'a çıksa monitoring nasıl olurdu?

Savunma değil açıklama

Mülakatçı bir tercihini sorguladığında hedef "kararımı savunayım" değil; bağlamı, alternatifi ve trade-off'u göstermek olmalıdır. Gerekirse "bugün yapsam bunu değiştirirdim" demek güçlüdür.

24. Final Kontrol Listesi

24.1 Repository

- ☐ Repo adı profesyonel ve açıklayıcı
- ☐ README ilk ekranda problemi anlatıyor
- ☐ Sonuçlar görünür
- ☐ Kurulum ve çalışma komutları doğru
- ☐ requirements/pyproject mevcut
- ☐ Kişisel path yok
- ☐ Secret yok
- ☐ Büyük veri/model yanlışlıkla commit edilmemiş
- ☐ Notebooklar baştan sona çalışıyor
- ☐ src/ kodu modüler
- ☐ En az kritik testler var
- ☐ README'de limitation var
- ☐ Görseller okunabilir
- ☐ Lisans/veri kaynağı belirtilmiş
- ☐ Fresh clone testi geçti

24.2 GitHub profili

- ☐ Profil açıklaması rol hedefini anlatıyor
- ☐ En güçlü 3-6 repo pinned
- ☐ Boş/yarım repolar öne çıkmıyor
- ☐ Repo isimleri tutarlı
- ☐ LinkedIn/portfolio bağlantıları çalışıyor
- ☐ Profil README gereksiz badge kalabalığı içermiyor

24.3 Yayınlama kriteri

Bir proje ne zaman "hazır"?

Bir yabancı kişi README'yi okuyup problemi anlayabiliyor, projeyi klonlayıp çalıştırabiliyor, sonuçların nasıl ölçüldüğünü görebiliyor ve sen projedeki her teknik kararı açıklayabiliyorsan yayınlanmaya hazırdır.

Modül Özeti

Profesyonel bir Data Science portföyü; algoritma sayısından çok problem çözme kalitesini, mühendislik disiplinini ve iletişimi görünür kılar. GitHub burada yalnızca dosya depolama alanı değildir: teknik kararların, çalışma biçiminin ve gelişim sürecinin kanıtıdır.

- Az sayıda fakat derin proje oluştur.
- README'yi case study gibi yaz.
- Notebook keşif için; tekrar kullanılan kod src/ için.
- Reproducibility ve testleri portföyün parçası yap.
- Commit geçmişini okunabilir tut.
- Veri, secret ve büyük artefact yönetimini bilinçli yap.
- Her projenin business kararını ve limitation'ını açıkla.
- En az bir projede demo/deployment göster.
- Projelerini 2 dakikalık ve 10 dakikalık formatta anlatabil.

Son çalışma görevi

Mevcut portföyündeki veya ilk oluşturacağın projeyi bu modüldeki 21. bölüm rubriğiyle değerlendir. En düşük üç alanı düzeltmeden yeni projeye geçme.